Automated Race Timing

ECMM427 – Group Development Project

The Running Group

May 2025

The Problem: Timing Races Accurately & Affordably

- Many amateur races rely on manual or expensive RFID-based timing systems.
- RFID systems are accurate but costly, require setup overhead, and are environmentally unfriendly.
- Manual timing (clipboards or tablet apps) is error-prone and labour-intensive.
- The goal: an **automated, affordable system** using cameras and computer vision to detect bib numbers and record finish times.



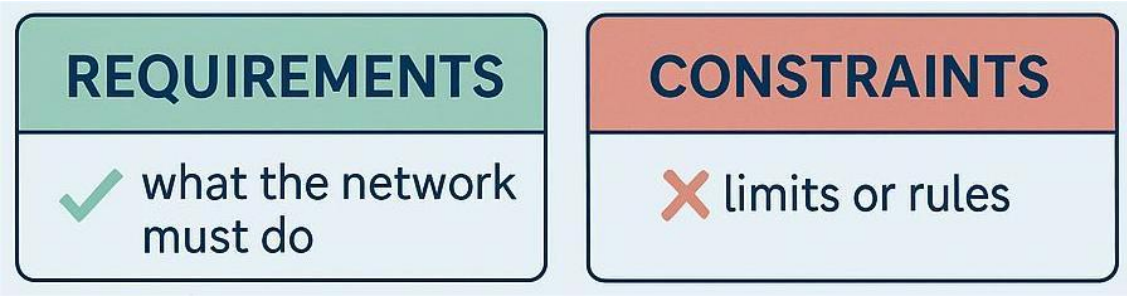
Race Timing Domain

- Must work for charity runs, park runs, and competitive races:
  - Suitable for medium to long distance events.
  - Can support large groups of participants.
- Needs high timing precision with an acceptable error margin of 1–2 seconds:
  - Ensures fair competition and trust in race results.
- Must be built for ease of use by non-technical event organisers:
  - Simple system with minimal setup requirements.
  - Intuitive user interface with clear, real-time feedback.



Requirements and Real-World Constraints

- Must be easy to setup.
- Must run on minimal hardware commonly available for both laptops and cameras.
- Must work in poor lighting, rain, and with cluttered visuals.
- Must accurately detect bibs, read numbers, and link runners with times.
- Must support a variety of categories such as age and gender.
- Must have real-time result uploads.
- Must have easy manual verification of results.



Development Process

Agile-Inspired Workflow

- Rapid prototyping for fast idea testing.
- GitHub issues track bugs and improvements.
- Iterative cycles enable continuous feedback.

Collaboration Tools

- GitHub Projects for managing code tasks.
- Trello boards for visual progress tracking.
- Tools that keep the team aligned and informed.

Validation with Real-World Data

- Early prototypes tested with sample race videos.
- Uncovered timing, detection, and hardware challenges.
- Feedback refined the system for real-world demands.



System Architecture Overview

- **Local Pipeline on Laptop:**
  - Video Input: Captures live footage during the race using a standard laptop camera or connected external devices.
  - Bib Detection: Utilizes computer vision algorithms to identify the bibs worn by runners, even in busy race environments.
  - Optical Character Recognition (OCR): Converts the visual bib numbers into digital text, ensuring data can be easily processed.
  - Timestamping: Accurately records the time each runner is detected, which is crucial for precise race timing.
- **Data Communication via REST API:**
  - Timing data is transmitted from the laptop to the website server using REST API calls ensuring secure, real-time data exchange.
- **Central Django Website:**
  - Leaderboard: Displays live race standings, updating in real-time as new timing data is received.
  - User Data Management: Handles registration details and participant profiles, facilitating a more personalised experience.
  - Event Information: Hosts comprehensive details about the event, including race categories and historical performance data.

Computer Vision Pipeline

- **Step 1: Runner and Bib Detection (YOLO)**
  - Uses the YOLO ("You Only Look Once") algorithm to process incoming video frames quickly and accurately.
  - Identifies not only the runners but also the bib regions they are wearing, even in dynamic or crowded scenes.
  - This fast detection is essential for real-time system performance during live events.
- **Step 2: OCR Extraction of Bib Number (EasyOCR)**
  - Once the bib is detected, the system applies EasyOCR to extract the numeric bib information.
  - Converts visual data into digital text by accurately reading numbers under varying conditions, such as different lighting or motion blur.
  - Ensures that each runner's bib can be linked precisely to their record.
- **Step 3: Timestamp Recording on Successful Final Read**
  - After confirming the bib number, the system records a timestamp corresponding to that moment.
  - This timestamp forms the basis of the race timing, ensuring an accurate log of each runner's performance.

Live Demo: Race Timing and Web Interface

- **Live Video Analysis:**
  - Stream a live feed with YOLO-based bib detection and OCR extraction.
  - Display real-time timestamping as runners are detected.
- **Web Dashboard Showcase:**
  - Demonstrate the manual verification interface for correcting misreads.
  - Show the user-friendly race display with live leaderboards and event updates.

Deployment and Setup

- **Website Hosting and Architecture:**
  - Built using the Django framework, the website utilises SQLite for data storage.
  - SQLite was chosen for its simplicity, making it ideally suited for small to medium-scale projects without the overhead of more complex database systems.
- **Local Application for Event Organisers:**
  - The local app runs on the organiser's laptop and can process live feeds from a standard webcam or an action cam feed from a phone.
  - This flexibility ensures that organisers can deploy the system with minimal investment and technical expertise.
- **University-Hosted Environment:**
  - The website is hosted by the University, allowing authorised users to access it via the secure University network.
  - The website can also be easily moved onto a different server environment.

Challenges We Tackled

- **OCR Errors in Dynamic Environments:**
  - Tackling inaccuracies caused by occlusions from arms and body movements.
  - Refining OCR settings to cope with motion blur and unexpected visual artefacts.
- **Non-Standard Bib Placements:**
  - Adapting our detection models to handle cases where bibs are not traditionally placed.
  - Addressing issues such as bibs being worn on the thigh or other unconventional locations.
- **Variability in Font Styles and Lighting Conditions:**
  - Mitigating the challenges posed by different font types and varying lighting across races.
- **Reducing False Positives and Misreads:**
  - Tuning detection thresholds and validation rules to ensure accurate recognition.

Future Extensions

- **Support for Multiple Cameras (Multi-Angle Detection):**
  - Integrate feeds from several cameras to capture the race from different perspectives.
  - Enhance detection accuracy by covering areas that might be occluded or missed by a single camera.
- **Expansion for Start Timing and Split Tracking:**
  - Incorporate start timing to record the precise moment the race commences.
  - Add split timing to monitor runner positions at key checkpoints throughout the course.
  - Provide organisers with more detailed performance analytics and real-time updates from throughout the event.
- **Migration to Mobile:**
  - Develop a mobile application optimised for smartphones and tablets.
  - Utilise the built-in cameras of mobile devices for more flexible, on-the-go race timing.

What We Learned

- Built a robust vision system that performs reliably even under imperfect conditions—handling low light, motion blur, and cluttered backgrounds.
- Gained valuable hands-on experience with Django for web development, YOLO for fast object detection, and OCR for accurate bib number extraction.
- Learned to balance engineering requirements with practical user needs and setup constraints, ensuring the system remains accessible for non-technical event organisers.

Conclusion

- **Low-Cost, Camera-Based Race Timing:**
  - Delivered a cost-effective solution that leverages readily available camera technology.
  - Minimises the need for expensive hardware without sacrificing performance.
- **Tailored for Race Organisers:**
  - Designed with usability in mind; optimised for race organisers rather than technical experts.
  - Features a straightforward setup process and intuitive controls to ease deployment at live events.
- **Real-Time, Scalable, and Efficient:**
  - Provides live race results to ensure timely updates for participants and organisers.
  - Requires minimal setup and can easily scale from small local runs to large, multi-participant events.